Introduction to Haskell Programming

Data structures – Stacks

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Stack

- Consider a `Stack` data type

Stack

- Consider a `Stack` data type
- A collection of `Int`s stacked one on top of the other

Stack

- Consider a `Stack` data type
- A collection of **`Int`**s stacked one on top of the other
- `push`: place an element on top of the stack

Stack

- Consider a `Stack` data type
- A collection of **`Int`**s stacked one on top of the other
- `push`: place an element on top of the stack
- `pop`: remove the topmost element of the stack

Stack

- Consider a `Stack` data type
- A collection of **`Int`**s stacked one on top of the other
- `push`: place an element on top of the stack
- `pop`: remove the topmost element of the stack
- Behaviour similar to lists: top of stack is head of list

Stack

- We could declare `Stack` to be a type synonym

```
type Stack = [Int]
push :: Int → Stack → Stack
push n st   = n:st
pop :: Stack → (Int, Stack)
pop []      = error "Empty stack"
pop (n:st)  = (n, st)
```

Stack

- We could declare `Stack` to be a type synonym

```
type Stack = [Int]
push :: Int → Stack → Stack
push n st   = n:st
pop :: Stack → (Int, Stack)
pop []      = error "Empty stack"
pop (n:st)  = (n, st)
```

- But this allows operations other than `push` and `pop`

```
take n st
st1 ++ st2
take (n-1) st ++ [x] ++ drop (n-1) st
```

Stack

- We want to allow only functions defined for stack

Stack

- We want to allow only functions defined for stack
- First step: make it a data type

```
data Stack = Stack [Int]
push :: Int → Stack → Stack
push x (Stack xs)   = Stack (x:xs)
pop :: Stack → (Int, Stack)
pop (Stack [])      = error "Empty stack"
pop (Stack (x:xs))  = (x, Stack xs)
```

Stack

- We want to allow only functions defined for stack
- First step: make it a data type

```
data Stack = Stack [Int]
push :: Int → Stack → Stack
push x (Stack xs)   = Stack (x:xs)
pop :: Stack → (Int, Stack)
pop (Stack [])      = error "Empty stack"
pop (Stack (x:xs))  = (x, Stack xs)
```

- If `st`, `st1`, `st2` are of type `Stack`, the following will not typecheck!

```
take n st
st1 ++ st2
take (n-1) st ++ [x] ++ drop (n-1) st
```

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
push :: a → Stack a → Stack a
pop :: Stack a → (a, Stack a)
empty :: Stack a
isEmpty :: Stack a → Bool

push x (Stack xs)   = Stack (x:xs)
pop (Stack [])      = error "Empty stack"
pop (Stack (x:xs))  = (x, Stack xs)
empty               = Stack []
isEmpty (Stack xs)  = null xs
```

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- `Stack` (occurring on the left) is not a type

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- `Stack` (occurring on the left) is not a type
- It is a **type constructor**

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- `Stack` (occurring on the left) is not a type
- It is a **type constructor**
- For any type `a`, `Stack a` is a type

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- `Stack` (occurring on the left) is not a type
- It is a **type constructor**
- For any type `a`, `Stack a` is a type
- The `Stack` on the right is a **value constructor** or a **data constructor**

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- `Stack` (occurring on the left) is not a type
- It is a **type constructor**
- For any type `a`, `Stack a` is a type
- The `Stack` on the right is a **value constructor** or a **data constructor**
- Given `xs :: [a]`, it constructs a value of type `Stack a`

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- `Stack` (occurring on the left) is not a type
- It is a **type constructor**
- For any type `a`, `Stack a` is a type
- The `Stack` on the right is a **value constructor** or a **data constructor**
- Given `xs :: [a]`, it constructs a value of type `Stack a`
  - `Stack xs :: Stack a`

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- Suppose we want to define `sumStack`:

```
sumStack (Stack xs) = sum xs
```

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- Suppose we want to define `sumStack`:

```
sumStack (Stack xs) = sum xs
```

- What is the type of `sumStack`?

Type parameters

- Polymorphic stack

```
data Stack a = Stack [a]
```

- Suppose we want to define `sumStack`:

```
sumStack (Stack xs) = sum xs
```

- What is the type of `sumStack`?
- Makes sense only when `xs` consists of numeric elements

```
sumStack :: Num a ⇒ Stack a → a
```

Applying Stacks – building expressions

- Recall code to build an expression from a list of tokens (presented in postfix order)

```
tokListToExp :: [Token] → Exp
tokListToExp toks                    = e where
    (e:_)                            = foldl step [] toks
    step :: [Exp] → Token → [Exp]
    step es             (Left n)     = Val n: es
    step (exp1:exp2:es) (Right op)   = op exp2 exp1: es
```

Applying Stacks – building expressions

- Recall code to build an expression from a list of tokens (presented in postfix order)

```
tokListToExp :: [Token] → Exp
tokListToExp toks                    = e where
    (e:_)                            = foldl step [] toks
    step :: [Exp] → Token → [Exp]
    step es             (Left n)     = Val n: es
    step (exp1:exp2:es) (Right op)   = op exp2 exp1: es
```

- Input is a list of tokens and the code uses a list of expressions

Applying Stacks – building expressions

- Recall code to build an expression from a list of tokens (presented in postfix order)

```
tokListToExp :: [Token] → Exp
tokListToExp toks                    = e where
    (e:_)                            = foldl step [] toks
    step :: [Exp] → Token → [Exp]
    step es             (Left n)     = Val n: es
    step (exp1:exp2:es) (Right op)   = op exp2 exp1: es
```

- Input is a list of tokens and the code uses a list of expressions
- More transparent to use a stack instead of a list!

Applying Stacks – building expressions

- Recall code to build an expression from a list of tokens (presented in postfix order)

```
tokListToExp :: [Token] → Exp
tokListToExp toks                    = e where
    (e:_)                            = foldl step [] toks
    step :: [Exp] → Token → [Exp]
    step es             (Left n)     = Val n: es
    step (exp1:exp2:es) (Right op)   = op exp2 exp1: es
```

- Input is a list of tokens and the code uses a list of expressions
- More transparent to use a stack instead of a list!
- Operations are restricted to push / pop

Applying Stacks – building expressions

- Input is a list of tokens and the code uses a list of expressions

Applying Stacks – building expressions

- Input is a list of tokens and the code uses a list of expressions
- More transparent to use a stack instead of a list!

Applying Stacks – building expressions

- Input is a list of tokens and the code uses a list of expressions
- More transparent to use a stack instead of a list!
- Operations are restricted to push / pop

```
tokListToExp :: [Token] → Exp
tokListToExp toks       = fst (pop st) where
    st                  = foldl step empty toks
    step :: Stack Exp → Token → Stack Exp
    step st (Left n)    = push (Val n) st
    step st (Right op)  = push (op exp2 exp1) st2 where
        (exp1, st1)     = pop st
        (exp2, st2)     = pop st1
```

Abstract data types (ADTs)

- If `st1 :: Stack a` and `st2 :: Stack a` ...

Abstract data types (ADTs)

- If `st1 :: Stack a` and `st2 :: Stack a` ...
- we cannot do `st1 ++ st2`

Abstract data types (ADTs)

- If `st1 :: Stack a` and `st2 :: Stack a` ...
- we cannot do `st1 ++ st2`
- But we can write the following function:

```
attach :: Stack a → Stack a → Stack a
attach (Stack xs) (Stack ys) = Stack (xs++ys)
```

Abstract data types (ADTs)

- If `st1 :: Stack a` and `st2 :: Stack a` ...
- we cannot do `st1 ++ st2`
- But we can write the following function:

```
attach :: Stack a → Stack a → Stack a
attach (Stack xs) (Stack ys) = Stack (xs++ys)
```

- So what have we gained by making it a data type?

Abstract data types (ADTs)

- If `st1 :: Stack a` and `st2 :: Stack a` ...
- we cannot do `st1 ++ st2`
- But we can write the following function:

```
attach :: Stack a → Stack a → Stack a
attach (Stack xs) (Stack ys) = Stack (xs++ys)
```

- So what have we gained by making it a data type?
- We need a way to provide a precise **interface** for `Stack`

Abstract data types (ADTs)

- If `st1 :: Stack a` and `st2 :: Stack a` ...
- we cannot do `st1 ++ st2`
- But we can write the following function:

```
attach :: Stack a → Stack a → Stack a
attach (Stack xs) (Stack ys) = Stack (xs++ys)
```

- So what have we gained by making it a data type?
- We need a way to provide a precise **interface** for `Stack`
- Hide the **implementation**

Abstract data types (ADTs)

- If `st1 :: Stack a` and `st2 :: Stack a` ...
- we cannot do `st1 ++ st2`
- But we can write the following function:

```
attach :: Stack a → Stack a → Stack a
attach (Stack xs) (Stack ys) = Stack (xs++ys)
```

- So what have we gained by making it a data type?
- We need a way to provide a precise **interface** for `Stack`
- Hide the **implementation**
- **Abstract data types** – realised via **modules**

Modules

- Ideally we would like the internal representation to be hidden from the world

Modules

- Ideally we would like the internal representation to be hidden from the world
- **Solution**: Create a `Stack` module

Modules

- Ideally we would like the internal representation to be hidden from the world
- **Solution**: Create a `Stack` module
- A **module** consists of functions that are related to each other

Modules

- Ideally we would like the internal representation to be hidden from the world
- **Solution**: Create a `Stack` module
- A **module** consists of functions that are related to each other
- The name of the file must match the name of the module

Modules

- Ideally we would like the internal representation to be hidden from the world
- **Solution**: Create a `Stack` module
- A **module** consists of functions that are related to each other
- The name of the file must match the name of the module
- The module can be used (imported) in any other file in the same directory

A Stack module

- The Stack module, saved in Stack.hs

```
module Stack(push, pop, empty, isEmpty) where
data Stack a = Stack [a]
push x (Stack xs) = Stack (x:xs)
pop (Stack (x:xs)) = (x, Stack xs)
empty = Stack []
isEmpty (Stack xs) = null xs
```

A `Stack` module

- The `Stack` module, saved in `Stack.hs`

```
module Stack(push, pop, empty, isEmpty) where
data Stack a = Stack [a]
push x (Stack xs) = Stack (x:xs)
pop (Stack (x:xs)) = (x, Stack xs)
empty = Stack []
isEmpty (Stack xs) = null xs
```

- The functions and types listed inside parentheses are **exported** – can be used outside the module

A `Stack` module

- The `Stack` module, saved in `Stack.hs`

```
module Stack(push, pop, empty, isEmpty) where
data Stack a = Stack [a]
push x (Stack xs) = Stack (x:xs)
pop (Stack (x:xs)) = (x, Stack xs)
empty = Stack []
isEmpty (Stack xs) = null xs
```

- The functions and types listed inside parentheses are **exported** – can be used outside the module
- `module Stack where` – everything defined in the module is exported

A Stack module

- The Stack module, saved in Stack.hs

```
module Stack(push, pop, empty, isEmpty) where
data Stack a = Stack [a]
push x (Stack xs) = Stack (x:xs)
pop (Stack (x:xs)) = (x, Stack xs)
empty = Stack []
isEmpty (Stack xs) = null xs
```

- The functions and types listed inside parentheses are **exported** – can be used outside the module
- `module Stack where` – everything defined in the module is exported
- `module Stack() where` – nothing is exported

Using the `Stack` module

- The following code is in `useStack.hs`, in the same directory

```
import Stack

myStack :: Stack Int
myStack = empty
```

Using the `Stack` module

- The following code is in `useStack.hs`, in the same directory

```
import Stack

myStack :: Stack Int
myStack = empty
```

- `import Stack` – imports everything exported by the `Stack` module

Using the `Stack` module

- The following code is in `useStack.hs`, in the same directory

```
import Stack

myStack :: Stack Int
myStack = empty
```

- `import Stack` – imports everything exported by the `Stack` module
- The line containing the type declaration for `myStack` does not not compile!

```
-- Not in scope: type constructor or class 'Stack'
```

Using the `Stack` module

- Need to export the type constructor

```
module Stack(Stack, push, pop, empty, isEmpty) where
data Stack a = Stack [a]
 ...
```

Using the `Stack` module

- Need to export the type constructor

```
module Stack(Stack, push, pop, empty, isEmpty) where
data Stack a = Stack [a]
...
```

- Now `useStack.hs` compiles

```
import Stack

myStack :: Stack Int
myStack = empty
```

Using the `Stack` module

- Can we do this in `useStack.hs`?

```
import Stack
newStack = Stack [0..9]
```

Using the `Stack` module

- Can we do this in `useStack.hs`?

```
import Stack
newStack = Stack [0..9]
```

- Does not compile!

Using the `Stack` module

- Can we do this in `useStack.hs`?

```
import Stack
newStack = Stack [0..9]
```

- Does not compile!
  - Following error if the type constructor is exported

```
-- Illegal term-level use of the type constructor ''Stack
```

Using the `Stack` module

- Can we do this in `useStack.hs`?

```
import Stack
newStack = Stack [0..9]
```

- Does not compile!
  - Following error if the type constructor is exported

```
-- Illegal term-level use of the type constructor ''Stack
```

  - Thinks we are using the type constructor to build a value!

Using the `Stack` module

- Can we do this in `useStack.hs`?

```
import Stack
newStack = Stack [0..9]
```

- Does not compile!
  - Following error if the type constructor is exported

```
-- Illegal term-level use of the type constructor ''Stack
```

  - Thinks we are using the type constructor to build a value!
  - Following error if type constructor is not exported

```
-- Data constructor not in scope: Stack :: [Integer] → t
```

Using the `Stack` module

- Can we do this in `useStack.hs`?

```
import Stack
newStack = Stack [0..9]
```

- Does not compile!
  - Following error if the type constructor is exported

```
-- Illegal term-level use of the type constructor ''Stack
```

  - Thinks we are using the type constructor to build a value!
  - Following error if type constructor is not exported

```
-- Data constructor not in scope: Stack :: [Integer] → t
```

- Ideally, we should export the data not export data constructors

Using the `Stack` module

- Can we do this in `useStack.hs`?

```
import Stack
newStack = Stack [0..9]
```

- Does not compile!
  - Following error if the type constructor is exported

```
-- Illegal term-level use of the type constructor ''Stack
```

  - Thinks we are using the type constructor to build a value!
  - Following error if type constructor is not exported

```
-- Data constructor not in scope: Stack :: [Integer] → t
```

- Ideally, we should export the data not export data constructors
- No one should be able to directly use the data constructor!

Exporting the constructors

- If we want the data constructors to be used directly ...

Exporting the constructors

- If we want the data constructors to be used directly ...
- we export the data constructors in the module

```
module Stack(Stack(Stack), push, pop, empty, isEmpty)
    where
data Stack a = Stack [a]
...
```

Exporting the constructors

- If we want the data constructors to be used directly ...
- we export the data constructors in the module

```
module Stack(Stack(Stack), push, pop, empty, isEmpty)
    where
data Stack a = Stack [a]
...
```

- In `Stack(Stack)`, the left `Stack` is the export of the type constructor

Exporting the constructors

- If we want the data constructors to be used directly ...
- we export the data constructors in the module

```
module Stack(Stack(Stack), push, pop, empty, isEmpty)
    where
data Stack a = Stack [a]
...
```

- In `Stack(Stack)`, the left `Stack` is the export of the type constructor
- The right `Stack` is the data constructor

Exporting the constructors

- If we want the data constructors to be used directly ...
- we export the data constructors in the module

```
module Stack(Stack(Stack), push, pop, empty, isEmpty)
    where
data Stack a = Stack [a]
...
```

- In `Stack(Stack)`, the left `Stack` is the export of the type constructor
- The right `Stack` is the data constructor
- In case there are many data constructors, we export them all by:

```
module Stack(Stack(..), push, pop, empty, isEmpty) where
```

Using the `Stack` module

- We create `module Stack` in `Stack.hs`

Using the `Stack` module

- We create `module Stack` in `Stack.hs`
  - exporting the type constructor but not the data constructor

Using the `Stack` module

- We create `module Stack` in `Stack.hs`
  - exporting the type constructor but not the data constructor
- We create `useStack.hs`, importing `Stack`

Using the `Stack` module

- We create `module Stack` in `Stack.hs`
  - exporting the type constructor but not the data constructor
- We create `useStack.hs`, importing `Stack`
- If we load `useStack.hs` in `ghci`, `Stack.hs` is also compiled

Using the `Stack` module

- We create `module Stack` in `Stack.hs`
  - exporting the type constructor but not the data constructor
- We create `useStack.hs`, importing `Stack`
- If we load `useStack.hs` in `ghci`, `Stack.hs` is also compiled
- But we can only use the exported functions

Using the `Stack` module

- We create `module Stack` in `Stack.hs`
  - exporting the type constructor but not the data constructor
- We create `useStack.hs`, importing `Stack`
- If we load `useStack.hs` in `ghci`, `Stack.hs` is also compiled
- But we can only use the exported functions
- One flaw in hiding our internal representation:

```
show (push 5 empty) = "Stack [5]"
```

Using the `Stack` module

- We create `module Stack` in `Stack.hs`
  - exporting the type constructor but not the data constructor
- We create `useStack.hs`, importing `Stack`
- If we load `useStack.hs` in `ghci`, `Stack.hs` is also compiled
- But we can only use the exported functions
- One flaw in hiding our internal representation:

```
show (push 5 empty) = "Stack [5]"
```

- There is a need for a custom **`show`**

A custom show

- Our original definition of `Stack`:

```
data Stack a = Stack [a]
    deriving (Eq, Show)
```

A custom `show`

- Our original definition of `Stack`:

```
data Stack a = Stack [a]
    deriving (Eq, Show)
```

- But the default **`show`** reveals the internal structure

A custom `show`

- Our original definition of `Stack`:

```
data Stack a = Stack [a]
    deriving (Eq, Show)
```

- But the default **`show`** reveals the internal structure
- We create a custom `Show` instance of `Stack a` as follows:

```
data Stack a = Stack [a]
    deriving Eq
instance Show a ⇒ Show (Stack a) where
    show (Stack l) = fancyShow l
```

A custom show

- Suppose we want `show (Stack [1,2,3]) = "1\n2\n3"`

A custom show

- Suppose we want `show (Stack [1,2,3]) = "1\n2\n3"`
- We create a custom `Show` instance as follows:

```
data Stack a = Stack [a]
    deriving Eq
instance Show a ⇒ Show (Stack a) where
    show (Stack l) = fancyShow l
fancyShow :: Show a ⇒ [a] → String
fancyShow = (intercalate " ⊢> ") . (map show)
```

The Stack module

```
module Stack(Stack, push, pop, top,
                fromList, empty, isEmpty) where
newtype Stack a = Stack [a] deriving Eq
instance Show a ⇒ Show (Stack a) where
    show (Stack xs) = intercalate " ⊢> " (map show xs)
push :: a → Stack a → Stack a
pop :: Stack a → (a, Stack a)
top :: Stack a → a
fromList :: [a] → Stack a
empty :: Stack a
isEmpty :: Stack a → Bool
```